

Weapon Detection and Classification in CCTV Surveillance: A Comparative Study of YOLO, RT-DETR and ResNet50

Joakim Möll
Jönköping University
Jönköping, Sweden
mojo21ti@student.ju.se

Jesper Claesson
Jönköping University
Jönköping, Sweden
clje22wn@student.ju.se

Abstract—Weapon detection in surveillance imagery is an important task for public safety, since the amount of video footage is too large for manual monitoring. In this work we compare five deep learning models on the HARIS dataset, including four YOLO generations (YOLOv5nu, YOLOv8n, YOLO11n, YOLO26n), the transformer based detector RT-DETR-L, and a ResNet50 classifier. All models were trained using transfer learning with the same hyperparameters. A small experiment showed that full fine-tuning worked better than freezing layers, so we used that approach for all main runs. The four YOLO nano models reached similar mAP50 scores between 83.4% and 84.4%, while RT-DETR-L scored lower, although it has around 15 times more parameters. YOLO26n reached its best performance in fewer epochs than the older versions. ResNet50 performed better than the YOLO models on pure image classification with 86.1% accuracy. Looking at the per-class results, the biggest source of errors is confusing weapons with background rather than confusing guns with knives. Overall, the newer YOLO versions did not outperform the older ones, and a two-proportion z-test confirmed that dedicated classifiers worked significantly better than detectors on the classification task ($p < 0.001$).

Index Terms—CCTV surveillance, object detection, ResNet, transfer learning, weapon detection, YOLO

I. BACKGROUND AND RELATED WORK

The increasing use of surveillance cameras in public spaces has created a growing need for automated threat detection systems. Since modern camera networks generate large amounts of video, manual monitoring is often impractical. Automated weapon detection using computer vision can help identifying potential threats more quickly, but the task remains challenging because of varying image quality and labeled data.

Recent advances in deep learning have made object detection systems more accurate and fast enough for real-time use. Among these approaches, the YOLO family of models have become a popular choice due to its balance between detection performance and inference speed. Bhatti et al. [1] compared several deep learning models for weapon detection in CCTV footage and found YOLO to provide one of the best trade-offs between speed and accuracy. Similar results were reported by León et al. [2], who applied a YOLOv5-based system for firearm detection in educational systems.

A closely related study by Fierro Silva et al. [5] compared five YOLO generations (YOLOv8s, YOLOv9s, YOLOv10s, YOLOv11s, and the newly introduced YOLOv26s) for weapon detection, evaluated on the Sohas Weapon Detection dataset. Their results provide one of few systematic multi-generation YOLO comparisons specifically for this task. However, their study is limited solely to YOLO architectures, and does not include a transformer-based detector or a dedicated image classifier for comparison, and it uses the small variants rather than the nano variants used in this work.

Transformer-based detection models have emerged as alternative to traditional CNN-based approaches. RT-DETR, introduced by Zhao et al. [3], was designed as a real-time transformer detector and achieved competitive results against YOLO on standard object detection benchmarks. Unlike YOLO, RT-DETR removes the need for non-maximum suppression (NMS) post-processing, which the authors claim introduces instability in both speed and accuracy due to sensitivity to threshold tuning. On the COCO benchmark, their RT-DETR-R50 model reached 53.1% AP at 108 FPS on a T4 GPU, which the authors reported as outperforming previous YOLO versions in both speed and accuracy at the time of publication.

Even though YOLO has been used a lot for weapon detection, most of the earlier work only looks at a single YOLO version or an older one, an observation also noted by Fierro Silva et al. [5], and it is not always clear whether the newer generations actually help on a smaller and more specialized CCTV dataset. There is also not much work that compares the YOLO models against a transformer detector like RT-DETR and a dedicated classifier like ResNet50 on the same weapon dataset. This is the gap we try to fill in this project, by comparing four YOLO generations, RT-DETR-L and ResNet50 under the same conditions on the HARIS weapon detection dataset [4]. The four YOLO generations and the ResNet50 classifier were trained directly on HARIS in this work, while results for RT-DETR-L are taken from previously published benchmark values on the same dataset.

II. DATA EXPLORATION

The HARIS dataset (Hostile Activity Recognition and Intelligent Surveillance) is a publicly available dataset hosted on Kaggle. The images are primarily collected from CCTV and surveillance-style footage, making it representative of real-world deployment conditions. The dataset contains approximately 9,700 training images and 1,800 validation images with two object classes: gun and knife.

Guns show up much more often than knives: 886 gun instances in the validation set versus 322 knife instances, close to a 2.75:1 split. This lines up with what you'd expect from real surveillance footage, where firearms simply appear more often than knives. But it also means the model has fewer knife examples to learn from during training, which is probably part of why we see lower recall on the smaller class later on.

Image sizes in the dataset are all over the place, from small crops around 250x150 pixels up to full HD and beyond, all saved as JPEG. That's not really surprising once you look at the filenames: prefixes like `ds_5__` and `ds_6__` show that HARIS was put together from several different source datasets rather than shot with one camera setup. This lines up with the "curated" label on the Kaggle page, where the authors mention the dataset was deduplicated and checked for leakage across splits. A few of the no-weapon images also look like they were pulled from video, going by filenames such as "`ds_9_hardneg__Normal_Videos_877_x264_f001476.jpg`", so part of the background class likely comes from surveillance video frames rather than photos. All images get resized to a fixed resolution before training anyway, so this doesn't cause any technical problems, but it does mean the model sees a pretty wide range of image quality and framing during training.

The dataset also contains subsets of close-up weapon images, as well as images containing no weapons, increasing the diversity of the dataset. The no-weapon images turn out to matter a lot. Without them, the model could just learn to flag anything unusual in the frame instead of learning what a weapon actually looks like. This connects directly to one of our main results: most errors came from the model missing weapons and calling them background, not from mixing up guns and knives. So separating weapons from background was the harder problem here, not telling the two weapon types apart. Mixing in close-up shots with normal CCTV-distance images adds to the difficulty too, since the model has to deal with a much wider range of object sizes and distances than a more uniform dataset would require.

III. METHOD DESCRIPTION

For object detection, four YOLO models were trained: YOLOv5n, YOLOv8n, YOLO11n and YOLO26n. All experiments were conducted using the Ultralytics framework, with model training on an NVIDIA A100 GPU. Detection models were initialized using weights from images pretrained on COCO dataset.

These models were selected to represent multiple generations of the YOLO family in comparable model sizes. The

nano variants were chosen because they are more suitable for a moderately sized dataset, and larger models require more computational resources. RT-DETR-L was not retrained in this study, the published benchmark values provided with the HARIS dataset were used for comparison.

We also include ResNet50 as a classification baseline, mainly to see how a model built just for image-level recognition stacks up against detectors when they're both reduced to a simple weapon-or-not decision. This separates two different questions: is a weapon present in the image at all, versus where exactly it is. In some security setups, just knowing a weapon is present might already be enough, so it's worth checking how well a plain classifier does on that simpler version of the problem.

For image classification, a binary ResNet50 classifier pre-trained on images from ImageNet was trained and fine-tuned to predict whether an image contains a weapon or not.

A. Network Architectures

Since we use three different model types in this study, we give a short description of how each one works.

YOLO (You Only Look Once) is a single-stage object detector. Unlike older two-stage detectors, it looks at the whole image once and predicts the bounding boxes and class probabilities directly, which is the main reason it is fast enough for real-time use. The network is made up of three parts: a backbone that extracts features from the image at different scales, a neck that combines these features, and a detection head that outputs the boxes and class scores. The four generations we use all follow this same general structure, and the main differences between them are improvements in the backbone, the neck, and the training settings rather than a completely new design.

ResNet50 is a convolutional neural network with 50 layers that is built from residual blocks. The main idea behind it is the residual connection, which takes the input of a block and adds it back to the output. This means each block only has to learn the difference, or residual, instead of the full mapping. The benefit is that the gradients can flow through very deep networks more easily, which is what makes it possible to train a network this deep without it getting worse, a problem that happens with plain deep CNNs. ResNet50 uses bottleneck blocks made of 1×1 , 3×3 and 1×1 convolutions. Because it is pretrained on ImageNet, it already has strong general features that transfer well to our task of deciding whether an image contains a weapon or not.

RT-DETR (Real-Time Detection Transformer) is a detector that is based on transformers, so it works in a pretty different way from YOLO. It builds on a model called DETR, which sees object detection as a set prediction problem. This means the model just predicts a fixed set of objects straight away, instead of going through lots of candidate boxes and scoring them. It does this with a transformer encoder and decoder that use attention. The decoder gets a number of object queries and turns each one into a final detection. The downside of the original DETR was that it was slow and hard to train,

so RT-DETR makes a few changes to speed it up. It uses a hybrid encoder that handles the features inside one scale separately from mixing the features across scales, which makes it cheaper to run. It also uses IoU (intersection over union)-aware query selection, which just means it picks better queries to start the decoder with, so it needs fewer of them and trains better. Together these changes are what let RT-DETR run in real time while still using a transformer.

B. Experiment Design

All detection models were trained using a batch size of 64, an input resolution of 640×640 pixels, and a maximum of 150 epochs. The best checkpoint was selected based on the mAP50-95 metric. Apart from model-specific defaults, the same training settings were used across all experiments.

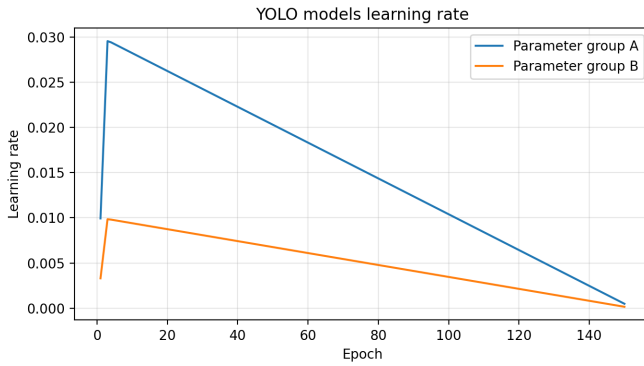


Figure 1. Learning rate over the 150 training epochs. Ultralytics selected the MuSGD optimizer automatically and applies two different rates to different parts of the network. Both warm up over the first epochs and then decay linearly.

Before the main experiments, a small study was conducted on layer freezing during transfer learning. Three configurations were compared: full fine-tuning, freezing the first 10 layers, and freezing the first 20 layers. Based on the results, full fine-tuning was used in all subsequent experiments.

For the classification task, we used a ResNet50 model pretrained on ImageNet, imported from the Keras library. Images were resized to 224×224 pixels, and followed a custom classification head with batch normalization, global average pooling, 40 % dropout, and a sigmoid output layer for binary classification. The last three layers were fully fine-tuned, based on a similar transfer learning test. Training used the Adam Optimizer with a constant learning rate of 0.0001.

IV. EVALUATION

A. Performance Metrics

The object detection models were evaluated using precision, recall, mAP50, and mAP50-95. Precision measures how often a predicted object is correct, while recall measures how many true objects are successfully detected. Both are important in a security setting, where missed detections and false alarms can have practical consequences.

mAP50 and mAP50-95 are standard object detection metrics that summarize detection quality across different intersection over union (IoU) thresholds, in the range 0.5 to 0.95.

B. Statistical Testing

To check whether the differences in classification accuracy between models are statistically meaningful, we used a two-proportion z-test, which compares the proportion of correct predictions from two models on the same test set. A p-value below 0.05 indicates that the difference is statistically significant at the 95% confidence level. We applied this test to two pairwise comparisons: ResNet50 against YOLO26n, and YOLO26n against YOLOv5nu. We also report the 95% confidence interval for the accuracy difference in each comparison.

V. RESULTS AND ANALYSIS

A. Freeze Layer Experiment

The results in Fig. 2 shows that full fine-tuning gave the highest mAP50-95 score of 54.9%, while freezing 10 layers reached 53.3%, only slightly lower. Freezing 20 layers caused a much bigger drop to 39.5%, around 15% lower than full fine-tuning. The small difference between freezing 0 and 10 layers shows that the early features from COCO transfer fairly well to HARIS, while the bigger drop at 20 layers suggests that the later layers need to adapt more to the CCTV style images. Based on this, we used full fine-tuning for the rest of our experiments.

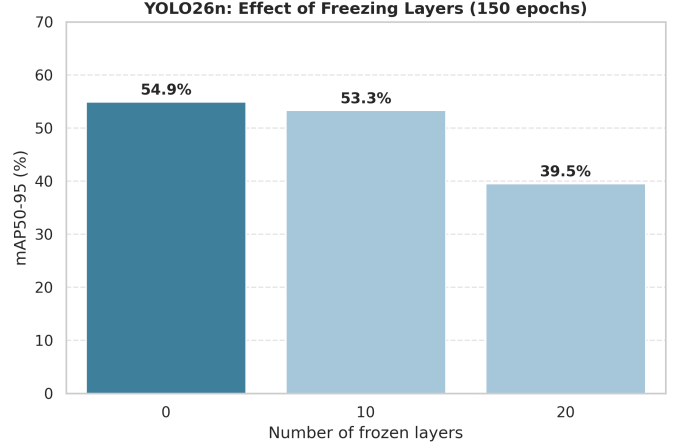


Figure 2. YOLO26n freeze-layer experiment comparing full fine-tuning with 10 and 20 frozen layers.

B. Object Detection Performance

The four YOLO models reach mAP50 scores between 81.1% and 84.9%, as shown in Table I. YOLO26n reaches 84.9% and YOLOv5nu 84.8%, followed by YOLOv8n at 82.6% and YOLO11n at 81.1%. All four generations perform at a similar level on this dataset, which supports the finding of Bhatti et al. [1] that YOLO is a reliable choice for CCTV weapon detection. The differences between them are under 4 percentage points, and since each model was trained once, we

cannot say whether that ordering would hold across repeated runs.

The differences are larger on mAP50-95, which measures bounding box quality across multiple IoU thresholds. YOLO26n reaches 55.5%, roughly 3 percentage points above YOLOv5nu (52.5%) and YOLOv8n (52.6%), and almost 5 above YOLO11n (50.8%). YOLO26n therefore produces better bounding boxes than the other generations, whether from tighter boxes, more accurate positioning, or both, while detecting a similar number of weapons.

The models differ in how they balance precision and recall. YOLOv5nu has the highest precision at 88.6% but lower recall at 79.4%. YOLO26n has the highest recall at 80.1% with precision at 86.9%, so it makes fewer misses at a small cost in false positives. YOLO11n is the lowest on both measures.

Table I

OBJECT DETECTION RESULTS ON THE TEST SET. BOLD VALUES INDICATE THE BEST PERFORMANCE PER METRIC.

Model	P	R	mAP50	mAP50-95	Ep.
YOLOv5nu	88.6%	79.4%	84.8%	52.5%	148
YOLOv8n	86.2%	77.4%	82.6%	52.6%	90
YOLO11n	82.8%	79.9%	81.1%	50.8%	124
YOLO26n	86.9%	80.1%	84.9%	55.5%	131
RT-DETR-L*	87.2%	70.4%	78.0%	47.6%	—

*Published benchmark values reported on the HARIS validation set. RT-DETR-L was not retrained or evaluated on the test split in this work, so these values are a rough reference point rather than a direct comparison.

RT-DETR-L behaves differently. Its precision (87.2%) is in the same range as the YOLO models, but its recall is much lower at 70.4%, which pulls down its overall mAP scores (mAP50 = 78.0%, mAP50-95 = 47.6%). It is more conservative in what it predicts, but that caution costs it more in missed weapons than it gains in avoided false positives. RT-DETR-L also has about 15 times more parameters than the YOLO nano models, around 32M compared to 2.5M, and still does not perform better on this dataset. This is different from what Zhao et al. [3] showed, where RT-DETR beat YOLO on COCO. Their results don't seem to carry over to a smaller and more specialized dataset like ours.

Convergence behavior: The Epoch column in Table I shows where each model hit its best mAP50-95, which is also the checkpoint we saved and evaluated. YOLOv8n got there first at epoch 90, while the rest peaked between 124 and 148. The mAP50-95 curve is pretty flat towards the end of training, so a small difference in a late epoch can move the peak by dozens of epochs. With one run per model there's no way to separate a real difference from noise.

C. Per-Class Analysis

Fig. 3 shows the confusion matrix for YOLO26n on the test set, using the default confidence threshold of 0.25. The matrix lets us see what kind of errors the model makes, more than what we get from the mAP scores alone. The model rarely confuses guns and knives with each other (0 guns was predicted as a knife, and 1 knife as gun).

Most errors are with the background class instead. Out of 438 gun instances, 98 (22.4%) were missed and classified as background, while 10 out of 160 knives (6.3%) were missed the same way. The model also made 83 false-positive gun detections and 23 false-positive knife detections on background regions.

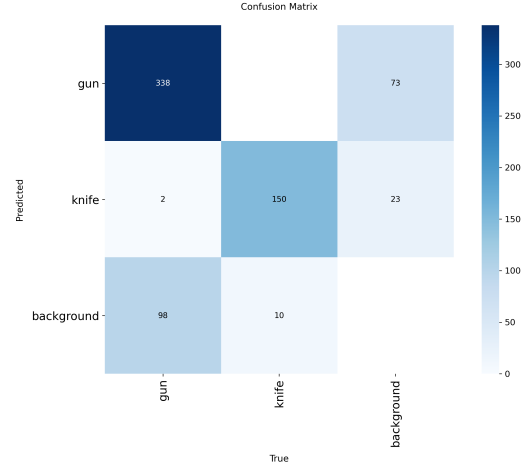


Figure 3. Confusion matrix for YOLO26n on the test set at confidence threshold 0.25. Rows are predicted classes and columns are ground truth.

Knife detection was actually more accurate than gun detection. The per-class metrics give 85.7% precision and 93.8% recall for knives (F1 = 0.896), compared to 82.2% and 77.2% for guns (F1 = 0.796). This was a bit unexpected since knives are usually thought to be harder because of their thin shape. One possible reason is that the knife images in HARIS often show the knife clearly in the foreground, while the gun images have more varied angles and are more often partly hidden behind a hand or clothing. Overall, the biggest source of error is not telling gun and knife apart but telling weapons apart from background.



Figure 4. YOLO26n detection of guns with confidence 0.5 and 0.9.



Figure 5. Two different YOLO26n errors. Left: an object detected as a gun (0.35) on an image containing no weapon. Right: a gun detected as a knife (0.41).

D. Example Detection Results on Validation Images

To illustrate how the detector behaves in practice, Fig. 4 shows example predictions produced by YOLO26n on validation images. The model outputs a bounding box around the detected object together with a confidence score representing the estimated probability that the object belongs to the predicted class. In both examples, the detector successfully localizes the weapons, with different confident scores (0.5 and 0.9), where the lower score corresponds to a more challenging image.

Fig. 5 shows two of the faulty classifications YOLO26n made. In the left image, the model detects a gun in an image that has no weapon in it, which is the most frequent kind of mistake the model makes. In the right image it detects a knife where the ground truth says gun, which only happened twice in the entire test set. Both predictions have low confidence (0.35 and 0.41), so raising the threshold above the default value of 0.25 would eliminate the faulty predictions, but it would also throw away correct detections that is close to the threshold, so that is a trade-off to consider.

E. Classification Comparison

We also looked at how the models do on plain binary classification. ResNet50 was trained directly for this, while for the YOLO models we converted their detections into per-image labels by counting TP, TN, FP, and FN values from each model’s confusion matrix at confidence threshold 0.25. The two setups are not fully comparable since ResNet predicts once per image while YOLO works per detection, but the comparison still gives a useful overview. The comparison should therefore be interpreted as a practical comparison of weapon-presence recognition rather than a direct comparison of model architecture. Table II summarizes the results.

Table II
CLASSIFICATION RESULTS ON THE TEST SET AT CONFIDENCE THRESHOLD 0.25. RESNET50 METRICS ARE COMPUTED PER IMAGE, WHILE YOLO METRICS ARE DERIVED FROM PER-INSTANCE CONFUSION MATRICES. BOLD VALUES INDICATE THE BEST PERFORMANCE PER METRIC.

Model	Precision	Recall	Accuracy
YOLOv5nu	78.5%	82.6%	73.4%
YOLOv8n	76.5%	82.9%	71.8%
YOLO11n	77.0%	82.4%	72.0%
YOLO26n	83.6%	81.9%	77.3%
ResNet50	82.5%	97.4%	86.1%

ResNet50 has the best accuracy at 86.1%, against 71.8 to 77.3% for the YOLO models. Much of that gap comes from the way the two are counted rather than the models being that different. ResNet50 makes one decision per image, so the worst it can do is be wrong once. The YOLO numbers come from per-instance counts, so a single image can rack up several false positives on its own. This comparison says more about weapon-presence recognition in practice than about one architecture being better than the other.

The precision and recall split is more interesting than the accuracy gap. ResNet50 has by far the highest recall at 97.4%, so it almost never misses a weapon, but its precision is the lowest of the five at 82.5%, meaning close to one in five of its positive predictions is a false alarm. YOLO26n is the opposite, with the highest precision at 83.6% and recall of 81.9%. ResNet50 leans towards saying "weapon" when it is unsure, which is partly down to the class imbalance in the training data, where about 74% of the images contain a weapon. Among the YOLO models, YOLO26n is again the strongest with 77.3% accuracy, which fits what we saw in the detection results. To check whether the differences are statistically significant, we ran two-proportion z-tests on the classification results. Table III summarizes them.

Table III
TWO-PROPORTION Z-TEST RESULTS FOR PAIRWISE CLASSIFICATION ACCURACY COMPARISONS ON THE TEST SET (N = 900 PER MODEL).

Comparison	Δ Acc.	z	p	95% CI
ResNet50 vs YOLO26n	8.8%	4.82	<0.001	[5.2, 12.3]%
YOLO26n vs YOLOv5nu	3.9%	1.92	0.055	[-0.1, 7.9]%

The difference between ResNet50 and YOLO26n is significant ($p < 0.001$), so the 8.8 percentage point gap is unlikely to be down to chance, even if part of it comes from the counting method rather than the models. The 3.9 point difference between YOLO26n and YOLOv5nu falls just short of the 0.05 threshold ($p = 0.055$), so we cannot claim a real difference between the two YOLO generations. This fits the detection results, where the four generations also landed within a few points of each other.

VI. DISCUSSION

The results suggest that newer YOLO versions do not automatically perform better on a smaller and more specialized dataset such as HARIS. The differences between the YOLO models were small, which may be because all models were already strong enough for the dataset or because single training runs do not capture the full variation between models.

RT-DETR-L performed worse than expected compared to the YOLO models. This differs from results on larger benchmark datasets, where RT-DETR has shown strong performance. One possible explanation is that transformer-based detectors may require more data to adapt well, while the lightweight YOLO models are easier to fine-tune on a specialized weapon dataset.

The per-class analysis showed that the main problem was not confusing guns with knives, but confusing weapons with background. This is important for real surveillance systems because missed weapons are dangerous, while too many false alarms can make the system less useful in practice.

The classification comparison also shows that detection and classification should not be treated as the same task. ResNet50 was better for image-level weapon recognition, but it cannot show where the weapon is located. A practical system could therefore combine a classifier for image-level screening with an object detector for localization.

VII. CONCLUSION

This report compared YOLOv5nu, YOLOv8n, YOLO11n, YOLO26n, RT-DETR-L, and ResNet50 on the HARIS weapon detection dataset. The YOLO nano models achieved very similar detection performance, with mAP50 scores around 84%. YOLO11n and YOLO26n reached slightly higher mAP50-95 scores, while YOLO26n reached its best checkpoint in fewer epochs.

RT-DETR-L did not outperform the YOLO models, despite having more parameters. It achieved high precision but lower recall, which made its overall detection performance weaker on this dataset.

For image-level classification, ResNet50 performed clearly better than the YOLO models, reaching 91.7% accuracy. The statistical test confirmed that the difference between ResNet50 and YOLO26n was significant. However, ResNet50 only predicts whether a weapon is present, while YOLO also localizes the weapon. Therefore, the best model choice depends on whether the goal is classification, localization, or both.

Future Work

A natural next step would be a hybrid setup that uses ResNet50 for image-level classification and YOLO for localization when a weapon is detected. This combines the strengths of both and could reduce false alarms.

A larger and more varied dataset would also help, especially with harder background images, since the per-class analysis showed that confusing weapons with background is the main source of errors. The HARIS dataset is fairly limited in size and contains mostly clear images, while real CCTV footage is often lower quality and has more unusual angles and occlusion.

Finally, it would be interesting to extend this to video footage, since most surveillance systems work with video and not single images. Looking across frames could help reduce false alarms, since a real weapon should show up consistently while a false detection usually does not.

REFERENCES

- [1] M. T. Bhatti, M. G. Khan, M. Aslam, and M. J. Fiaz, "Weapon detection in real-time CCTV videos using deep learning," *IEEE Access*, vol. 9, pp. 34366–34382, 2021.
- [2] F.T. León *et al.*, "Early Warning System for Firearm Detection on University Campuses Using Computer Vision," in *Journal of Internet Services and Information Security (JISIS)*, vol. 16, no. 1, pp. 127–142, 2026.
- [3] Y. Zhao *et al.*, "DETRs beat YOLOs on real-time object detection," in *2024 IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, Seattle, WA, USA, 2024, pp. 16965–16974.
- [4] T. Alshuaibi, A. AlGhamdi, A. Alsaid, H. Alzahrani, and K. Alomair, "HARIS weapon detection dataset (curated)," Kaggle, 2026. [Online]. Available: <https://www.kaggle.com/datasets/turkikhalidalshuaibi/haris-weapon-detection>
- [5] C. J. Fierro Silva, C. Del-Valle-Soto, C. Bran, and J. Varela-Aldás, "Comparative analysis of previous YOLO detectors and YOLOv26s for real-time weapon detection in video surveillance," *Front. Comput. Sci.*, vol. 8, art. 1789702, 2026, doi: 10.3389/fcomp.2026.1789702.